5 ANTICIPATING AND DEALING WITH ERRORS

OBJECTIVES

- Explain techniques for preventing errors being introduced into your solution.
- Discuss how to mitigate the effects of errors that are introduced.
- Introduce a systematic approach to testing via top-down and bottom-up strategies.
- Show how to test individual parts of a solution in isolation.
- Discuss a number of debugging strategies.
- Explain how to manage the bugs found in your solution.

COMING TO TERMS WITH BUGS

> I well remember when ... the realization came over me with full force that a good part of the remainder of my life was going to be spent in finding errors in my own programs.
>
> (Wilkes, 1985)

You should get used to the idea that solutions you create will not work flawlessly. They will contain bugs somewhere, despite your very best efforts. Even the most talented programmers and computer scientists in the world produce software that contains bugs, even if some are loath to admit it. A bug can be as simple as a typo in a formula or as disastrous as a complete misunderstanding of the original problem definition.

Bug: a fault in a solution that can cause erroneous behaviour.

Bugs can be caught and eliminated at any stage in the development process, but the earlier you find and fix them the better. You'll find it much easier to change something when it exists only as an idea rather than after it's been implemented. Think of the effort a writer needs to put into fixing a plot hole when it's found during the planning phase as opposed to in the finished draft. Or think of the cost of recalling and fixing an error in a smartphone after millions have been sold. Therefore, designing out the bugs is a good practice. This chapter will examine such practices.

Error: any behaviour observed when executing a solution that does not match expected behaviour.

Some bugs will only become apparent after your solution (or parts of it) has been put into effect. For those cases, you need different approaches, namely testing and debugging. These will also be discussed in this chapter.

DESIGNING OUT THE BUGS

This section examines the various types of bugs that can cause erroneous behaviour.

Typos

During the early planning phases, your solution exists in various design forms: documents, notes, diagrams, algorithm outlines and so on. The simplest way of preventing errors at this stage is to look for elementary mistakes, in both the problem definition and your solution.

Typos include:

- spelling errors;
- incorrect capitalisations;
- missing words;
- wrong words (for example, 'phase' instead of 'phrase');
- numbered lists with incorrect numbering, and so on.

Such mistakes may often be trivial, but any of them could cause problems.

Poor grammar and ambiguities

Like typos, some types of grammatical error are quite easy to locate and correct. Its easy to find the mistake in this sentence, right?[31] However, some grammatical errors are quite subtle. For example:

> After having a coin inserted, the user will be able to push the turnstile.

What or who exactly is having a coin inserted here?[32] Statements like these need rephrasing so they can only be interpreted one way. You should therefore **not** underestimate the usefulness of good writing skills.

Still, ambiguities can appear in sentences that are grammatically fine. For example:

> After the login process fails a number of times in a row, the incident should be reported in an email to the security administrator.

The intention behind this statement is fine, but what does 'a number of times' actually mean? Two, five, a hundred? It may be permissible to decide on the exact number at a later stage, but in general you should nail down such details as early as possible.

Another type of ambiguity that can potentially cause problems is when the (apparently) same concept is referred to using different terms. This may seem innocent enough, but the implicit differences might suggest there are several concepts that need distinguishing or that an abstraction can be created. For example, if your solution makes various references to a 'smartphone', 'tablet', and 'handheld device' all in the same context, you should question whether those terms can really be used interchangeably or whether you should treat them differently.[33]

Keep a glossary of key terms used in your solution and stick to it.

Inconsistencies

Eliminating ambiguities helps to counter another source of bugs, namely inconsistencies. An inconsistency arises when parts of a solution separately make statements that make no sense when considered together. These bugs are harder to locate because each part of the inconsistency might appear in very different contexts.

Consider the example of the login process (above) rewritten as follows:

> After the login process fails five times in a row, this incident should be reported in an email to the security administrator.

Also imagine that a different part of the solution contains the following statement:

> After the login process fails three times in a row, the login screen should automatically lock and refuse further login attempts.

Here we have an inconsistency. If a login attempt fails three times consecutively, then the user is unable to make further login attempts. Consequently, they can never actually perform five login attempts in a row, and so the first statement is redundant.

Logical and mathematical errors

Simple bugs might be found in logical expressions. For example, the following algorithm is supposed to print the maximum value of two numbers, x and y.

```
If x > y, then print y
Otherwise if y > x, then print x
```

This actually contains two bugs. One should be obvious: the algorithm prints the minimum instead of the maximum. The second is slightly more subtle: what happens if both numbers are equal? You would expect a value to be printed, but in fact, nothing is printed at all. This example demonstrates that errors can result from expressions that

are valid (that is, they can be evaluated without a problem) but whose semantic meaning is incorrect.

Similarly, mathematical errors can also result from seemingly valid expressions. An in-car computer that evaluates the car's efficiency based on the trip counter might use a formula like this:

$$e = \frac{m}{g}$$

where efficiency (e) is the miles travelled (m), divided by the gallons of fuel consumed (g). This will mostly work fine, except that when the trip counter is reset, the gallons consumed becomes zero and dividing by zero is mathematically invalid. The solution would have to take this special case into account.

Being exhaustive

Chapter 2 pointed out some of the limitations of human–computer interaction. Here's a reminder: computers have no common sense; they will not try to fill in any gaps in your instructions, and they can't conduct a dialogue with you to pin down exactly what you meant to say.

The path that a computer takes through an algorithm varies at different times. It all depends on the parameters passed, as well as other changeable factors in the environment. Therefore, algorithms should be exhaustive and contain instructions for all conceivable outcomes. If you fail to account for any of them, the computer's behaviour becomes unpredictable: it might take an unwanted action or it might do nothing when it should in fact do something.

These kinds of mistakes are easily made and are common with beginners (Pea et al., 1987; Pane et al., 2001). Even the professionals make such mistakes, more than they'd care to admit. Writing a robust algorithm that accounts for all possible pathways (both desired and undesired) takes experience, but you can learn some general principles as well as how to avoid some common errors:

- To reiterate some advice from Chapter 2: think critically. Once you've written a sequence of instructions that you expect to be followed, start to question what might go wrong. Examine each step and ask yourself how it might be possible for things to turn out differently from how you wish. If necessary, add instructions that take action in case of those undesired outcomes.
- Look especially at conditionals:
  - If you have a sequence of instructions in the form 'if X is the case, then do Y', ask yourself what happens if X is **not** the case? Sometimes a contingency is required. Therefore, always be mindful of Z – as in, 'if X is the case, then do Y, else do Z'.
  - Compound conditionals can be troublesome (for example, if A and not B or C...). Think about all the possible combinations and be aware that the number of combinations increases exponentially as a conditional increases in size.

  - An old and trusted piece of advice says 'bugs lurk in corners and congregate at boundaries' (Beizer, 1990). Take this example: 'if score < 50, then performance is OK; if score > 50, then performance is good'. What's missing? It doesn't take account of the case when the score is exactly 50. The advice is telling you to take extra care when testing ranges and boundaries.
- Also look especially at loops:
  - If a loop can be exited early, insert an explicit instruction to do so at the correct point.
  - If the rest of a loop iteration can be skipped and the computer can move onto the next iteration, again, insert an explicit instruction to do so at the correct point.
  - Make **absolutely** sure the loop cannot go on endlessly.

General tips

As the author of the solution, you're not an impartial observer. You will find it hard to see any problems that exist, partly because you're too wrapped up in the details of how it **should** work, and partly because you don't want to think of your beloved creation as flawed. You may also, without realising it, have made implicit assumptions that prove unwarranted. It's therefore a good idea to find a friend or colleague who will look over your proposed solution and perform these checks in addition to you.

It was recommended in Chapter 3 that you approach problem-solving iteratively. Now is a good time to reiterate that advice. When designing the solution, do so in small steps. Work on the solution in stages, rounding off each iteration by applying these checks.

Do your utmost to keep different parts of your solution self-contained by minimising the number of relationships between them.[34] This should localise the effects of any errors and help to prevent knock-on effects.

Finally, if the problem definition came to you from someone else, call on that person to verify your solution design. Misunderstandings between the solution designer and the 'client' are a notorious source of problems.

MITIGATING ERRORS

So far, this chapter has examined some **preventative** actions you can take to stop bugs making their way into your solution. The bad news is that your finished solution will very likely contain bugs, despite your best efforts. The good news is that steps exist for you to take against errors. The following sections examine what you can do to **mitigate** their effects.

Getting defensive

One way to minimise the effects of errors is to anticipate where a problem might occur and then put some 'protective barriers' in place. Computer science calls this **defensive programming**. The approach verifies that certain conditions are as expected before carrying out the main business of the solution. A failure to meet those conditions could either indicate anything from a warning to a potential system failure.

As with other practices in CT, we can see this same approach in other disciplines too. Physical systems where safety is an important issue are designed defensively. That is, they are safe by default. The default state of an elevator without power is to have its brakes **on**. To make the elevator move, power must be supplied to the brakes to switch them **off**. If the elevator loses power, it reverts to its default, powerless state with brakes applied. Hence, when something goes wrong, the people inside the elevator are automatically kept safe without any need for action. In essence, the elevator needs explicit 'permission' to carry out its normal business.

You can make your solution behave analogously to the elevator. Before the solution takes a possibly erroneous step, make it verify a set of preconditions before the 'normal business' can proceed. For example, if a step divides two numbers, you know that it will fail if the divisor[35] is zero. Therefore, you can create a precondition that checks this value. If it isn't zero, then normal business may proceed.

Reacting to problems

What to do in the case of a failed precondition depends on the context, and, in particular, on the consequences of an error. If the severity is low, the action taken could be something as minor as displaying a message. If the severity is high and risks causing real damage, the system should enter into some kind of emergency mode or even shut down.

For example, normal business for a drink vending machine is to read the value of inserted money and serve the customer their choice of drink. But several things could go wrong and the programmed responses vary:

- The customer might insert unreadable or foreign money. The machine recovers simply by returning the money (ideally with a helpful message to the customer).
- A particularly smart vending machine might have the ability to detect when a cup has fallen over on the dispensing tray. In this case, the machine should refuse to continue functioning until the cup is righted, after which it returns to normal business.
- A leak in the vending machine's water supply is a severe problem, since it risks water running onto the floor or into the machine's electrics. If a leak is detected, the prudent thing is probably for the machine to shut down.

Other general considerations when reacting to errors include:

- whether the system is interactive or not (which dictates whether or not it can ask the user what to do);
- whether the system is transactional or not. If it is, then it's possible to undo the changes made during a defective step.

Checking user input

In cases where your solution asks for input from a user, a little 'healthy paranoia' goes a long way. Humans make mistakes all the time (and malevolent users do so intentionally),

so it's best to assume that your user will enter faulty input at some point. Think of all the ways an input could be wrong:

- If your solution expects a number, check that it really is a number before doing anything with it. Entering '90' instead of '90' is an easy enough mistake to make.
- Similarly, if your solution requires input within a certain range, check it after it's been entered. Making sure an account number is the expected eight digits in length is quick and easy. All you have to do is count the number of digits in the input and ensure it's the right amount.
- Incorrect formats: phone numbers, website URLs, email addresses, dates and times are all examples of data that **must** match a specific format.

Being proactive and guarding against unwanted behaviour is essential. However, some bugs will slip in despite your efforts.

TESTING

This section describes testing, a method for locating hidden bugs in a fully or partially working system.

Goal

The aim of testing is to use the actual system you've produced in order to verify whether it performs as expected and to identify specific areas where it fails. Therefore, testing has to wait until your solution has reached some kind of working state. It could be fully working, meaning you can test the whole system, or it could be partially working, allowing you to test individual parts.

In order to perform testing effectively, you need to actually know what the solution aims to do. This makes the advice from Chapter 3 – clearly define the goal of your solution – essential. You gain little by verifying that a system performs as expected when you have poorly defined expectations. The solution's goal serves as a guide for testing.

Approach

It's very important in testing to be **systematic**. Ad hoc testing using no particular approach is easy to do, but the only thing you're likely to learn is that something is wrong somewhere. Being systematic will help you to localise the problem and find out exactly what is wrong and where.

You can choose between two approaches to testing, depending on what you want to achieve. The first is **top-down** testing, where you test the solution as a whole to ensure that it works. This is most effective at finding design flaws and verifying that the system hangs together well. The second approach is **bottom-up**, which requires you to begin

by testing the smallest parts of the solution individually. This allows you to verify that they correctly fulfil their own obligations within the whole system and to show that your solution is built on solid foundations.

Table 5.1 Advantages and disadvantages of top-down and bottom-up testing

	Advantages	Disadvantages
Top-down	• Effective for finding design flaws. • Can be encouraging to test a full system, even if it is incomplete or contains bugs.	• Difficult to apply when the system is incomplete. • Requires you to use placeholders[36] for the unfinished parts to simulate their behaviour. • Harder to localise bugs when errors occur.
Bottom-up	• Easy to apply in early stages of development. • Effective for localising problems.	• Requires you to simulate the controlling parts of the solution.[37] • Doesn't expose design flaws until much later.

If you can, you should combine these approaches when testing, thereby gaining the best of both worlds. When you begin to implement your solution, you can start by putting together a skeletal structure filled with placeholders. This should be executable, even though it might not actually do anything useful yet. Subsequently, you can start to implement the individual pieces, replacing each placeholder with a functioning part that you can immediately test in isolation (this is bottom-up testing). At the same time, because you have a working skeletal structure, you can test the whole system to make sure it still functions as expected (top-down testing).

Testing individual parts

If you followed the advice in Chapters 3 and 4 so that your solution exists in reasonably independent pieces, testing will be made simpler. By testing only small parts of the solution at once, you make localising the problem easier. Tests that exercise only individual parts are called **unit tests**. However, testing individual parts can be tricky and merits some explanation. The trickiness comes because the individual parts of your solution normally operate as part of a larger whole and so assume that the rest of the system is present and correct. That's not the case when they're being tested in isolation, so you have to simulate whatever things the part is dependent on.

For example, some components expect to be given parameters as part of their normal work, so to test them you'll need to provide some dummy data that simulate normal operation. In the ideal case, you would come up with every conceivable different value

that could possibly be provided and then run a test for each one. If you could do that, you would be able to give a solid guarantee that the component works as expected. However, that's often unfeasible because there are too many possible variants.

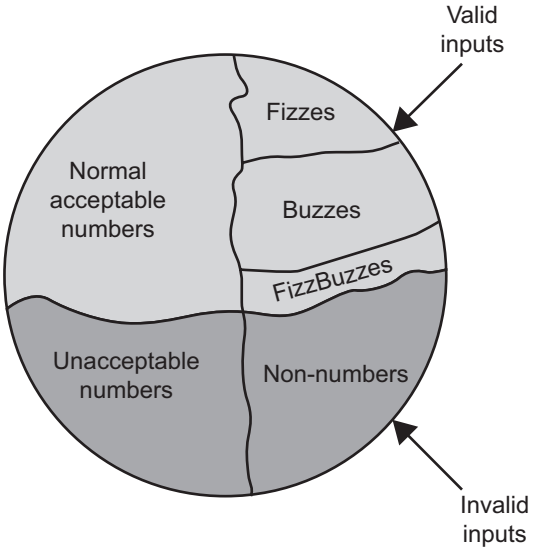
To illustrate this, I'll use the example of the FizzBuzz game. In this game, players take turns to count upwards from 1, replacing any number divisible by 3 with the word 'Fizz' and any number divisible by 5 with 'Buzz'. Numbers divisible by both 3 and 5 are replaced with 'FizzBuzz'. A player loses if they say a number when they should have said one of the words. The first few numbers in the correct sequence are:

> 1, 2, Fizz, 4, Buzz, Fizz, 7, 8, Fizz, Buzz, 11, Fizz, 13, 14, FizzBuzz, 16, 17, Fizz, 19, Buzz ...

A computer-based FizzBuzz game requires a subroutine that calculates the correct response for a given number. To test that this subroutine works, you could try it out for the first 20 values. That would tell you for certain whether it works for those values. But does it work for 21? Technically, you don't know for certain. The problem is, no matter what how large the set of test values, more numbers exist that you could include. Furthermore, the larger the set of test values, the longer it takes to carry out all the tests.

Despite this, you can be systematic. A component in a solution usually dictates requirements regarding its input data and you can use this information to make your testing systematic. Each requirement corresponds to a distinct set of inputs called equivalence classes. The thing that relates all inputs in each class is that they elicit the same type of response from the solution. Consequently, you can choose one (or several) pieces of representative test data from each group to stand in for all other values in that class. You can do the same with input data that contradict each requirement.

Figure 5.1 All inputs divided into equivalence classes



For the FizzBuzz game, we can divide all input into several equivalence classes (Figure 5.1), choose a representative value from each class, give them to the subroutine and then verify that in each case the correct response is returned. The classes include:

- **Normal acceptable numbers (for example, 8):** represents all acceptable numbers not divisible by 3 or 5. The subroutine should return the same number we gave it. If this number works, it implies that all other numbers in this class also work.
- **Fizzes (for example, 3):** represents all numbers divisible by three. Subroutine should return 'Fizz'.
- **Buzzes (for example, 10):** represents all numbers divisible by five. Subroutine should return 'Buzz'.
- **FizzBuzzes (for example, 30):** represents all numbers divisible by both three and five. Subroutine should return 'FizzBuzz'.
- **Unacceptable numbers (for example, -10):** numbers out of range should be rejected.
- **Non-numbers (for example, VII):** Arabic numerals only, please. A computer regards this as a piece of text and should reject it.

And, because 'bugs lurk in corners and congregate at boundaries' (Beizer, 1990):

- **Acceptable boundary (for example, 1):** this should work.
- **Unacceptable boundary (for example, 0):** this should be rejected.

Testing is a great place to benefit from another pair of eyeballs. You should record all your tests in a document (a test plan).[38] Make them clear and easy to understand. Then, give that plan to a third party and have them test your solution. Another perspective will help you to find problems that you, on your own, might miss.

DEBUGGING

> You know my methods. Apply them.
>
> Sherlock Holmes in *The Sign of Four* (Conan Doyle, 1890)

Inevitably, at some point during testing or real-world use of your solution, a puzzling error will occur. It will be unclear what particular step of the solution caused the problem, but there's no need to panic. Once again, you can be systematic in finding and fixing, that is **debugging**, the error. There are methods you can apply.

Take inspiration from Sherlock Holmes, who treated detection as a science. Your goal is to explain the facts (that is, the specific error(s) that occurred) and reconstruct the chain of events that led up to them. You need to look for clues and use them to make deductions. These will help you to build a hypothesis that might explain the problem. Once you have that, you can put it to the test. If your test is successful, you will have found the error(s) in your solution and can proceed to think about a fix.

That is the approach **in general**. In actuality, there is no single guaranteed method for debugging, but numerous different strategies are available (listed below) for building and testing your hypothesis. You'll need some creativity and imagination to help you. Naturally, you'll get better at debugging the more you do it.

Be ruthless with hunches

If you have a hunch about the problem, try it. You created the system, so you should have a good feel for how it works and how it might go wrong.

However, if your hunch is proved incorrect, **discard it immediately!** Don't get overly attached to your favourite explanation.

Respect Occam's Razor

You may well have several ideas about what could be causing the problem. Some might be simple, others complex. Occam's Razor is a principle in problem-solving that tells us the simpler explanation – the one that requires us to make the fewest assumptions – is most likely the correct one.

In other words, resist the urge to dream up complicated explanations. Try the simple ones first and eliminate those.

Divide and conquer

If you're unsure where the problem lies, make life easier for yourself by reducing the number of places you need to search. If, when designing your solution, you broke down your problem using a divide-and-conquer approach, you can now take advantage of that by using the same approach to debugging.

Divide and conquer allows you to eliminate whole areas of your system as suspects at once. The layout of your system – the components and their relationships – should resemble a tree structure. As your system executes, control is passed from component to component. Consequently, if the problem you're investigating arises **before** control ever passes to a certain component, then you can eliminate that component as a cause of your problem. Furthermore, any component that is **only** invoked by that eliminated component can itself be eliminated also.

Change one thing at a time

Good debugging is like good experimentation. When a scientist carries out an experiment, they're looking for the cause of something. In their case, they vary some factors and analyse the consequences to establish a link between cause and effect. However, a scientist takes extra care to vary only one factor at a time. After all, if they tweak three factors at once and then successfully detect an effect, how can they know which of the three factors was responsible?

Apply that same lesson during your debugging. If you think a change can fix your problem, apply it and then test it. If it didn't work, undo the change and put the system back to its original state before you try anything else.

Logging

At some point you'll likely observe that your solution does something unexpected; either it does something you didn't intend it to or it **doesn't** do something it should have. You need therefore to investigate which instructions in your system are executed at what point. But since execution happens inside the computer, you're not in a position to 'see' it.

However, you can get something of an overview by inserting an additional logging instruction somewhere in your algorithm. When this instruction is executed it will flag it up (for example, by printing out a message). The logging instruction acts like a turnstile; every time the computer 'goes through', this fact is reported. For example, you might expect that the computer goes through a particular loop five times, so you add a log instruction inside the loop that does nothing other than print out the word 'Loop'. If, when you run the system, the final result is:

```
Loop
Loop
Loop
Loop
```

you know that the loop was executed only four times.

You can do similar things if you're suspicious of certain values. You might observe that your solution does something unwanted, making you suspect that an incorrect value is the cause. It's not unusual, for example, that a block of instructions fails to trigger when you expected.

```
if x > 10, then do stuff
```

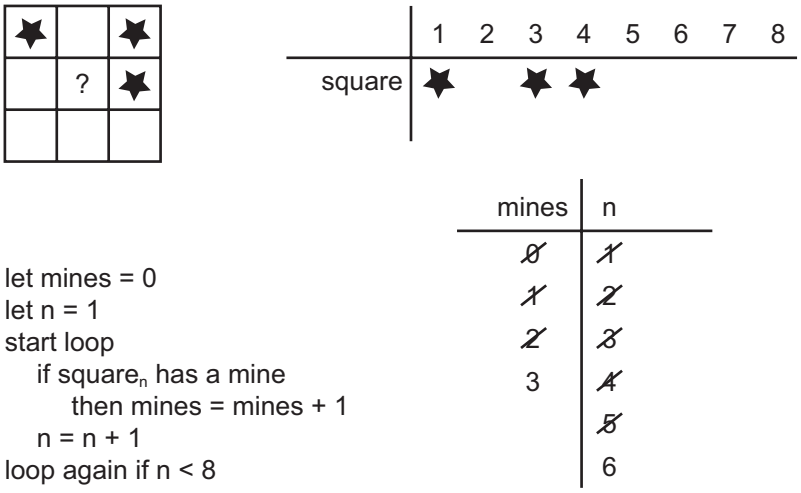
If, when you run the program, the instructions represented by '`stuff`' are not carried out, what would you suspect causes the problem? A strong possibility is that the variable *x* is actually less than or equal to 10 when this line is carried out. You could confirm this by adding a log instruction (immediately before the if-statement) that prints out the value of *x*. Once you have established the actual value, you can then investigate why *x* ended up with this particular value.

Tracing

Sometimes the only way to diagnose a problem is to go line by line through each instruction and see for yourself what the computer is actually doing.

You can do this yourself by hand, pretending to be the computer. You will need to go through each instruction, execute it, and keep track of the changes to any data the algorithm maintains. Figure 5.2 shows an example algorithm from a Minesweeper game where the game board is divided into squares. Each square may or may not be adjacent to a hidden mine. The game calculates the number of mines in the immediate vicinity of a square by going clockwise through each adjacent square and counting up the number of mines. Figure 5.2 shows someone partway through tracing, having gone through the 'repeat-until' loop six times already. They've executed each line and kept track of each change to the two variables.

Figure 5.2 Partway through tracing a Minesweeper algorithm



Once your system is implemented for a computer, you can also perform tracing using a **debugger**. This is a computer-aided tool that executes a program under your careful control. Essentially, it reveals to you what's going on in the computer's mind as it runs your solution. You can use a debugger to step through instructions one by one and print out the current state of various data, although many debuggers have many more advanced features. Use of debuggers is explored in more detail in Chapter 12.

Explain the problem aloud

Whenever an unexplained error stumps you, try explaining the problem vocally.[39] Step through what is wrong, what you've already tried and what you haven't tried. You might be surprised by how quickly a new explanation occurs to you after putting the problem into words.

This can be effective even when you do it alone (I won't tell anyone you talk to yourself). But for best results, explain your problem to someone else. It doesn't matter if the other person doesn't fully understand your solution. In fact, it's better if they don't because the other person might unwittingly help you to respect Occam's Razor. The problem might be so simple that it needs a 'silly' question to expose the simple mistake you've made.

YOU CAN'T HAVE EVERYTHING: DECIDING WHICH ERRORS TO FIX

Every piece of non-trivial software in the world has bugs in it. Your solution will be no different. You can keep testing and debugging until the cows come home, but there will always be flaws.

This is an important thing to acknowledge. It forces us to stop embarking on an endless pursuit of perfection, and instead manage the list of problems. If not all flaws can be

repaired, then we need a way to choose which ones to fix. Software developers have built up extensive experience in the management of errors. This section examines some of that experience.

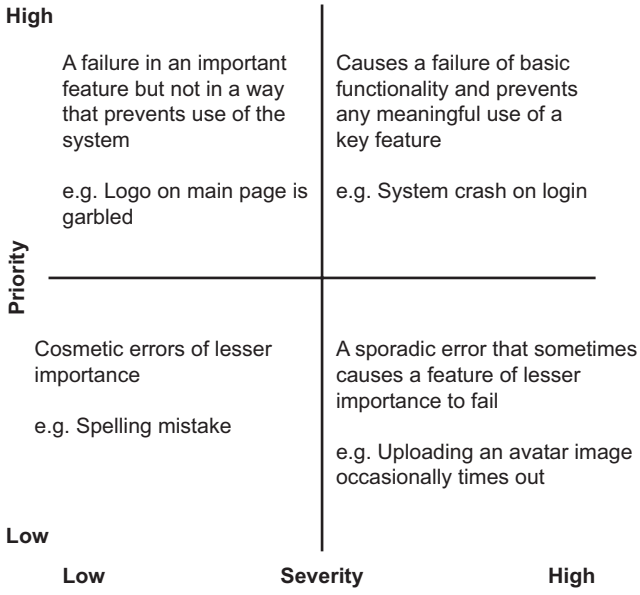
The first question to ask about an error is: what is its level of **severity**? This describes how strongly the error prevents proper usage of the system. For example, a bug that causes the system to fail has a high severity; when a logo is coloured slightly incorrectly, that's a low severity. Severity values are typically 'Minor', 'Moderate', 'Major' and 'Critical'.

The next question to ask is: what is its **priority**? This is a slightly more complex question to answer because you have to take several factors into account:

- **Frequency:** the more often the error occurs, the higher priority the bug should get.
- **Feature value:** if the bug affects an unimportant feature, it can get a low priority. If a key feature is broken, the bug should get a high priority.
- **Scheduling:** estimate how long it will take to fix an error and ask if it's worth the time. Is it worth fixing a rarely occurring, minor flaw if it's estimated to take a whole week? On the other hand, almost any amount of effort spent fixing a regularly occurring system failure is time well spent.

Priority values are typically 'Low', 'Medium' and 'High' as shown in Figure 5.3.

Figure 5.3 Priority severity matrix



Putting bugs on an axis, as in Figure 5.3, helps us to sort them and choose which ones to fix next. It's clear that bugs that are high-priority/high-severity should be ranked as most important, while low-priority/low-severity should be ranked last. For the two middle categories, I would suggest that high-priority/low-severity problems are tackled earlier.

SUMMARY

Bugs are mistakes that can cause errors in your solution. Numerous techniques exist either to prevent bugs making it into your solution or to deal with them after they've made their way in.

Preventing bugs from getting into your solution means careful design effort. The earlier bugs are found, the cheaper they are to repair.

Mitigating errors means anticipating and controlling their potential consequences. Programming defensively is a great way to prevent bugs causing havoc.

Hunting for bugs and removing them is done by testing and debugging. Testing lets you search for bugs and both top-down and bottom-up strategies offer systematic ways to approach testing. Debugging using logs or tracing helps you track down the ultimate cause of errors.

EXERCISES

EXERCISE 1

Mark the following statements as true or false:

A. Errors can result from statements that are grammatically valid.
B. The exit condition of a loop is evaluated after each statement inside the loop is executed.
C. Defensive programming assumes that code is safe until proven otherwise.
D. Bottom-up testing is an effective way of finding design flaws.
E. An equivalence class groups all inputs to an algorithm that elicit the same type of response.

EXERCISE 2

Explain the difference between a bug and an error.

EXERCISE 3

An 'if–then' conditional tells the computer to carry out some instructions if a certain condition is met. How could you modify this form to carry out instructions if the condition is **not** met?

EXERCISE 4

The algorithm in the Minesweeper example (Figure 5.2) contains an error. Find it and fix it.

The squares as they are configured in the example won't necessarily expose the error. Try debugging with the mines in different positions.

EXERCISE 5

Write an algorithm for the FizzBuzz game that analyses an input number and returns the correct response. If the number isn't in the expected range, the algorithm should return a suitable error message.